Cloud Workload Forecasting and Capacity Planning Report

1 Executive Summary

This report asks how predictable the supplied Azure VM workload is over a two-hour horizon and what an infrastructure team should do with the forecast. The release contains 10,000 VMs observed at 134 five-minute timestamps. The two required targets are the number of active VMs and the mean VM-level CPU utilization.

Active VM count is constant. Every VM has a valid reading at every timestamp, so `active_vm_count` is exactly 10,000 throughout. Persistence forecasts it with zero error. This is a property of the complete-trace selection, not evidence that churn is unimportant in production.

Aggregate CPU is low and stable. In the 110-point training window the mean is 9.41%, the standard deviation is 0.19 percentage points (pp) and the range is 9.11–9.91%. Lag-1 autocorrelation is about 0.67.

Model. I selected damped persistence for CPU. It starts from the last observation and decays toward the history mean. Eleven candidates were compared with expanding-window rolling-origin validation (7 origins, 24-step horizon). Damped persistence had the lowest MAE (0.203) and RMSE (0.242), 23.7% better than persistence in MAE. Its edge over the training-mean forecast is only 3.1% in MAE, and no model wins at every origin, so the ranking among the top models is not strong evidence.

Holdout. On the untouched final 24 points, damped persistence has MAE 0.153 and RMSE 0.183, against 0.147 and 0.185 for persistence and 0.175 and 0.211 for the training mean. Persistence and damped persistence are effectively tied. The damped forecast was too high in 75% of holdout steps, mostly early in the window, because it reverted toward the mean faster than the load recovered.

Recommendation. Refresh the damped forecast every five minutes and add a one-sided safety margin of about 0.41 pp, estimated from validation residuals. Trigger pre-scaling from the upper band, not the point forecast. The short trace does not justify aggressive capacity reduction.

2 Data Understanding and Preprocessing

2.1 Target construction

The CPU file has 1,340,000 rows: 10,000 VMs $\times$ 134 timestamps, from 0 to 39,900 seconds. The metadata file has one row per VM (lifecycle, workload category, core bucket, memory bucket). The pipeline (`src/cwfc/data.py`) removes missing or non-finite `avg_cpu`, keeps values in $[0, 100]$, drops duplicate `(vm_id, timestamp)` pairs, and groups by timestamp:

- `active_vm_count`: number of distinct valid VM identifiers;
- `aggregate_cpu_utilization`: arithmetic mean of valid VM-level `avg_cpu`.

The full target series is built first, and only then split chronologically: the first 110 timestamps are training and the last 24 are the two-hour holdout.

Cadence. Modeling uses the native five-minute cadence, so no aggregation or mapping back to the required timestamps was needed.

2.2 Quality checks

There are no duplicate `(vm_id, timestamp)` pairs, no null readings and no out-of-range values. Every timestamp is present and every VM trace is complete, so no imputation or resampling was applied. Silently

filling gaps would have changed both targets. All VMs were created at timestamp 0 and none was deleted inside the window, which explains the constant count and limits what the data can say about churn.

3 Workload Analysis

3.1 Aggregate behavior

The training CPU series has mean 9.414%, median 9.378%, standard deviation 0.194 pp and a last value of 9.165%. The series shows a hump of about 0.5 pp above its average, which no simple baseline follows well. A peak is defined as a value at or above the training-only 90th percentile (9.668%), giving 11 peak timestamps. The threshold uses no holdout data. Because the mean is low, a 0.4 pp error is about 4% of the level.

3.2 Segment differences

Segment	Mean CPU (%)
Interactive	11.49
Unknown (729 VMs)	10.23
Delay-insensitive	8.24

Table 1: Mean CPU by workload category.

The 2-core bucket holds 7,534 VMs and sets the overall level. The 4-core bucket (1,908 VMs) averages 7.93%. By memory, the 4-GB group averages 11.34% and the 2-GB group 8.78%. These differences matter operationally because a stable aggregate can hide a rising interactive or high-memory subpopulation.

4 Forecasting Approach

4.1 Candidates and features

All models use only the target history available at the forecast origin; there are no calendar or exogenous features. Candidates were:

- persistence (last value);
- training-history mean;
- moving averages with windows 3, 6 and 12 (15, 30 and 60 minutes);
- seasonal-naive with a 12-step period (reference only, since 11 hours cannot establish a seasonal period);
- simple exponential smoothing (SES);
- recursive Ridge regression on lagged and rolling-mean features, with $\alpha \in \{1, 10, 100\}$;
- damped persistence.

Availability assumption: the history mean, the lag-1 autocorrelation and every lag or rolling feature are computed only from observations up to the origin.

4.2 Selected model

For `active_vm_count`, persistence is used. For aggregate CPU the selected model is damped persistence:

$$\hat{y}(h) = \bar{y} + \phi^{\,h}\,(y_{\text{last}} - \bar{y}), \tag{1}$$

where $\bar{y}$ is the history mean and ϕ is the lag-1 autocorrelation of the history, clipped to $[0, 1]$. Forecasts are clipped to valid ranges. The selection rule was fixed before the Ridge run: a candidate replaces damped persistence only if its rolling-origin MAE is lower and its peak MAE is not worse. No candidate met this, so no further models were tested.

5 Validation Methodology

Random row splits would leak future workload states into training, so all evaluation is chronological. Within the 110-point training window I use expanding-window rolling-origin validation: minimum history 48 points (4 hours), horizon 24 points (2 hours), origin step 6 points (30 minutes). This gives 7 origins and 168 forecast points and mimics production: forecast the next two hours from everything seen so far.

The windows overlap (6-point step, 24-point horizon), so the origins are not independent and small differences should not be over-read. Metrics are MAE, RMSE, relative MAE improvement over a reference model and peak-period MAE. For percentage errors I use sMAPE with a small epsilon in the denominator, because MAPE is unstable near zero. The final 24 holdout points were not used for fitting, feature selection, tuning or model selection.

6 Results Versus Baseline

6.1 Rolling-origin validation

For `active_vm_count`, all seven baseline candidates evaluated on this target (persistence, mean, three moving averages, damped persistence, seasonal-naive) have zero MAE and RMSE. Persistence is retained as the simplest choice. Table 2 gives the CPU results.

Model	MAE	RMSE	vs. persistence	Peak MAE
Damped persistence	**0.2025**	**0.2419**	+23.7%	0.396
Training mean	0.2090	0.2468	+21.3%	0.416
Ridge, $\alpha = 100$	0.2163	0.2566	+18.5%	0.356
Moving average (3)	0.2439	0.2953	+8.2%	0.337
Moving average (6)	0.2532	0.3051	+4.6%	0.349
SES	0.2546	0.3051	+4.1%	0.345
Persistence (last value)	0.2655	0.3174	0.0%	**0.246**
Moving average (12)	0.2688	0.3121	−1.2%	0.379
Ridge, $\alpha = 1$	0.2745	0.3302	−3.4%	0.349
Seasonal-naive (12), reference	0.2809	0.3293	−5.8%	0.399
Ridge, $\alpha = 10$	0.2870	0.3525	−8.1%	0.357

Table 2: Rolling-origin validation for aggregate CPU (pp). "vs. persistence" is relative MAE improvement; peak MAE uses points at or above the 9.668% training 90th percentile.

Three points qualify these results.

- **The top of the ranking is within noise.** Damped persistence beats the training mean by only 3.1% in MAE (2.0% in RMSE) and has the lowest error at just 2 of 7 origins (the training mean also leads at 2). Five different models win at least one origin.
- **Complexity does not help.** Ridge with weak regularization is worse than persistence, which is likely overfitting on roughly 36–98 training rows. Strong regularization helps only by pulling the forecast toward the mean, which the training-mean model already does. Models that revert to the long-run average do best.
- **Peak evidence is thin.** Persistence has the best peak MAE (0.246 vs. 0.396 for damped persistence), but the 30 validation peak points come from only 11 distinct timestamps.

6.2 Final 24-point holdout

Model	MAE	RMSE	MAE vs. persistence	MAE vs. mean
Damped persistence (final)	0.1533	0.1832	−4.5%	+12.2%
Persistence	0.1468	0.1848	0.0%	+15.9%
Training mean	0.1746	0.2105	−18.9%	0.0%

Table 3: Holdout results for aggregate CPU (pp). Active-count MAE and RMSE are 0.

Persistence has slightly lower holdout MAE and damped persistence slightly lower RMSE (0.8%), so I treat them as tied. The holdout gap to persistence is much smaller than the validation gap (23.7%), which shows why a single window should not decide model choice. No holdout point reaches the training 90th percentile, so peak error cannot be evaluated on the holdout.

7 Error and Failure Analysis

7.1 Horizon dependence

Step (minutes)	Persistence	Training mean	Moving avg. (3)	Damped
1 (5)	0.121	0.166	0.090	0.103
3 (15)	0.117	0.225	0.157	0.202
6 (30)	0.150	0.224	0.146	0.216
12 (60)	0.242	0.219	0.250	0.218
18 (90)	0.267	0.167	0.236	0.167
24 (120)	0.257	0.132	0.252	0.132

Table 4: Validation MAE by forecast step (pp).

Recent-value methods win at short steps and the long-run mean wins at the end; the curves cross near step 12. Damped persistence is good at step 1 and at steps 18–24 but clearly worse than persistence at steps 3–6 (0.202 vs. 0.117 at step 3). The cause is that its decay is faster than the series' real memory. With $\phi \approx 0.67$ the weight on the last value is about 0.09 after 30 minutes, while the observed autocorrelation at 30 minutes is still 0.48. A model with slower decay is worth testing.

7.2 Holdout error structure

The main holdout failure is early over-forecasting, not under-forecasting. The forecast was above the actual value in 75% of steps (mean signed error +0.124 pp). The five largest errors (0.24–0.35 pp) all occur in steps 2–7 and all are over-predictions (forecast 9.30–9.40% vs. actual 8.98–9.16%).

Steps	Mean abs. error	Mean signed error
1–6	0.238	+0.238
7–12	0.171	+0.171
13–18	0.165	+0.091
19–24	0.040	−0.004

Table 5: Holdout error by six-step block (pp; signed error = forecast − actual).

The model is not biased overall. It moved back toward the training mean (9.41%) faster than the load recovered from the low last-observed value of 9.165%. By the last block, forecast and actual nearly coincide. The model was not changed after this analysis. Persistence stays closer in this window because the series was already near its representative level.

7.3 Other limits

Exploratory only: in the top quarter of holdout values (6 points, cutoff taken from the holdout itself), the mean-reverting forecasts were far more accurate than persistence (MAE 0.059 vs. 0.309). Combined with the validation peaks, where persistence was best, this suggests the best method depends on regime. Inside a hump, recent values win; when load returns toward average, mean reversion wins.

The model sees only aggregate history. It cannot anticipate new deployments, class transitions or incidents, and the constant VM count means churn could not be stress-tested. The validation residuals come from a short, stable trace, so their quantiles may understate risk under a regime change.

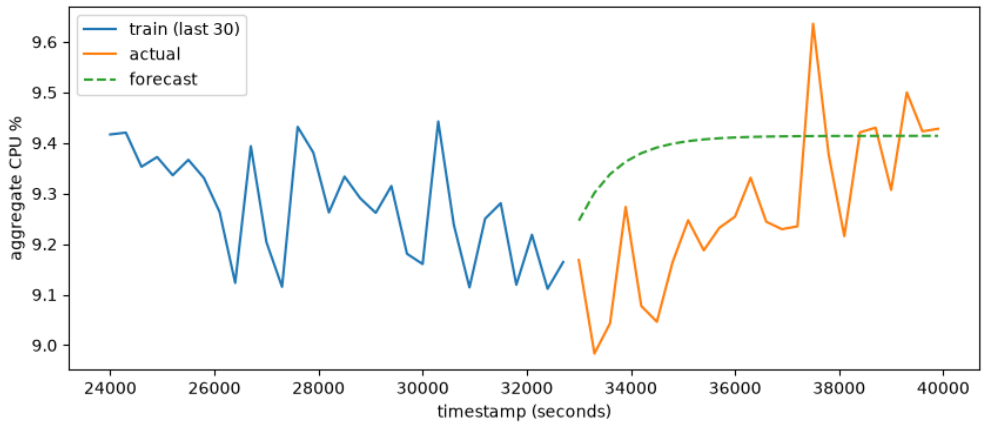


Figure 1: Last 30 training points, holdout actuals and the damped-persistence forecast.

8 Capacity-Planning Recommendation

Use the damped point forecast for the next two hours, refreshed every five minutes, plus a one-sided margin from validation under-prediction errors. The 90th percentile of (actual − forecast) is about 0.41 pp:

$$\text{Planned CPU} = \text{Forecast CPU} + 0.41 \text{ pp}, \tag{2}$$

clipped to $[0, 100]$. A central 90% validation band is about -0.335 to $+0.456$ pp; the upper offset is what matters for provisioning. Policy:

1. **Normal operation:** provision to the forecast plus 0.41 pp.
2. **Peak-risk alert:** review or pre-scale when the upper forecast band reaches the organization's CPU threshold.
3. **Receding demand:** scale in only after two consecutive refreshed forecasts fall below the scale-in threshold, to prevent oscillation.
4. **Population monitoring:** alert independently on active-VM-count changes; the constant count is a dataset property, not a safe assumption for a live fleet.

Trade-off. Extra headroom raises cost and leaves resources idle; too little risks throttling, latency or failed placement during an upward move. Because validation peak errors exceed average errors, the policy favors availability. In the holdout the forecast was already above the actual in 75% of steps, so this window would have been over-provisioned rather than under-provisioned; the margin's coverage was not tested there. The safety quantile can be lowered if the business accepts more risk, but that choice should rest on cost-of-error estimates, not accuracy alone.

9 Limitations and Future Work

- **Short trace.** About 11.1 hours cannot reveal daily or weekly seasonality, and the seasonal-naive result is only a reference.
- **No churn.** The complete-trace selection removes arrivals and departures from the count target.
- **Few, overlapping origins.** Seven overlapping origins and 11 distinct peak timestamps make model rankings and peak conclusions tentative.
- **Mean CPU only.** Mean CPU can hide a small set of saturated VMs; memory, disk, network and SLOs are not modeled.
- **Uncertainty.** The band is an empirical error quantile with no guaranteed coverage under distribution shift.

Future work: test a slower-decay model (for example ARMA or ETS with longer memory), extend the trace to several days, add deployment and lifecycle signals, forecast segments and high-percentile VM CPU in a hierarchy, calibrate intervals with conformal or rolling-residual methods, and repeat selection on a longer backtest with cost-weighted metrics that penalize under-provisioning more.